

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**



Кафедра программного обеспечения вычислительной техники и
автоматизированных систем

Лабораторная работа № 8
по дисциплине: Архитектура виртуальных систем
тема: **«Способы вызова ассемблерных подпрограмм в языках
высокого уровня»**

Выполнила: ст. группы ВТ-231
Масленников Даниил Александрович

Проверили:
Бондаренко Татьяна Владимировна
Осипов Олег Васильевич

Белгород, 2025 г.

Цель работы: изучение способов вызова подпрограмм, написанных на разных языках программирования посредством dll-библиотек.

Задание:

1. Написать и отладить подпрограммы на `masm32` в разных стилях вызова для решения задачи соответствующего варианта. Глобальные переменные в подпрограммах использовать не разрешается. Если нужна дополнительная память, выделять её в стеке.
2. Подпрограммы собрать и скомпилировать в виде `dll`-библиотеки. Библиотека должна содержать:
 - подпрограммы в стилях `stdcall`, `cdecl`, `fastcall`, написанные на ассемблере без явного перечисления аргументов в заголовке;
 - Подпрограммы в стилях `stdcall`, `cdecl`, написанные, наоборот, с перечислением аргументов в заголовке подпрограммы.
3. Подключить все подпрограммы из `dll`-библиотеки к проектам на `C#` и `C++` статическим и динамическим способом. Убедиться в правильности вызова всех подпрограмм.
4. Написать подпрограмму для решения задачи варианта с использованием ассемблерной вставки на языке `C++`.
5. Написать подпрограммы для решения задачи варианта с использованием обычного высокоуровневого языка `C#` и `C++` (или любого другого).
6. Сравнить скорость выполнения полученных подпрограмм на одних и тех же тестовых данных. Для сравнения выбрать: подпрограмму на ассемблере в `masm32` (какую-нибудь одну из пяти), вызываемую из программы на языке `C++` или `C#`; подпрограмму на `C#`; подпрограмму на `C++`; подпрограмму на `C++` с использованием ассемблерной вставки. Построить на одной плоскости графики зависимости времени выполнения подпрограмм от длины массивов (не менее 10 точек для каждой подпрограммы). Для замера лучше передавать в подпрограммы массивы большой длины. Время замерять в миллисекундах с помощью API-функции `GetTickCount()`. Проверить, что подпрограммы при одинаковых тестовых данных

выдают одинаковый результат. Для заполнения массивов использовать генератор случайных чисел.

7. В отчёт включить весь исходный код и графики.
8. Сделать выводы по работе.

Варианты 8 - 16

Из массива *a* длиной *length* скопировать отрицательные числа в массив *neg_res*, положительные – в массив *pos_res*. Под массивы *neg_res* и *pos_res* в основной программе зарезервировать памяти столько, сколько занимает массив *a*. Полученные массивы отсортировать. Количество отрицательных чисел записать в выходной параметр *neg_count*, положительных – в выходной параметр *pos_count*. Исходный массив *a* оставить без изменений. Для удобства, можно реализовать в dll-библиотеке отдельную процедуру для сортировки одного массива.

Пример: *a* = {1, 3, 4, -5, 7, -2, -1, 3, 5, -5}, *length* = 10;
pos_res = {1, 3, 3, 4, 5, 7} (сортировка по не убыванию);
neg_res = {-1, -2, -5, -5} (сортировка по не возрастанию);
pos_count = 6;
neg_count = 4.

14

Сортировка методом вставок по не убыванию.

int sort (int* pos_count, double* neg_res, double* a, int length, double* pos_res).

neg_count вернуть.

Выполнение работы:

.386

.model flat, stdcall

option casemap:none

public sort_std

public sort_c

public sort_f

public sort_stdcall

public sort_c_arg

.code

sort_worker PROC a_:DWORD, neg_res_:DWORD, pos_res_:DWORD,
length_:DWORD, pos_count_ptr_:DWORD

push ebp

mov ebp, esp

sub esp, 12

push ebx

push esi

push edi

mov dword ptr [ebp-4], 0

mov dword ptr [ebp-8], 0

mov dword ptr [ebp-12], 0

mov esi, a_

mov edi, pos_res_

split_loop:

mov eax, [ebp-12]

mov ecx, length_

cmp eax, ecx

jge split_done

mov ecx, eax

shl ecx, 3

lea edx, [esi + ecx]

mov eax, [edx+4]

test eax, 80000000h

jnz store_neg

mov eax, [edx]

```
mov     ecx, [edx+4]

mov     ebx, [ebp-4]
mov     edx, ebx
shl     edx, 3
lea     edx, [edi + edx]

mov     [edx],    eax
mov     [edx+4],  ecx

inc     ebx
mov     [ebp-4],  ebx
jmp     next_i
```

store_neg:

```
mov     eax, [edx]
mov     ecx, [edx+4]

mov     ebx, [ebp-8]
mov     edx, ebx
shl     edx, 3
mov     ebx, neg_res_
lea     edx, [ebx + edx]

mov     [edx],    eax
mov     [edx+4],  ecx

mov     ebx, [ebp-8]
inc     ebx
mov     [ebp-8],  ebx
```

next_i:

```
mov     eax, [ebp-12]
```

```
inc    eax
mov     [ebp-12], eax
jmp     split_loop
```

split_done:

```
mov     eax, [ebp-4]
cmp     eax, 1
jle     skip_pos_sort
```

```
mov     dword ptr [ebp-12], 1
```

pos_outer:

```
mov     eax, [ebp-12]
cmp     eax, [ebp-4]
jge     pos_done

mov     ecx, eax
shl     ecx, 3
lea     edx, [edi + ecx]
fld     qword ptr [edx]

mov     ebx, eax
dec     ebx
```

pos_inner:

```
cmp     ebx, -1
jl      pos_insert

mov     ecx, ebx
shl     ecx, 3
lea     edx, [edi + ecx]

fld     qword ptr [edx]
```

```
fcomp    st(1)
fstsw    ax
sahf
jbe      pos_insert
```

```
mov      eax, [edx]
mov      ecx, [edx+4]
mov      [edx+8], eax
mov      [edx+12], ecx
```

```
dec      ebx
jmp      pos_inner
```

pos_insert:

```
inc      ebx
mov      ecx, ebx
shl      ecx, 3
lea      edx, [edi + ecx]
fstp     qword ptr [edx]
```

```
mov      eax, [ebp-12]
inc      eax
mov      [ebp-12], eax
jmp      pos_outer
```

pos_done:

skip_pos_sort:

```
mov      esi, neg_res_
mov      eax, [ebp-8]
cmp      eax, 1
jle      skip_neg_sort
```

```
mov      dword ptr [ebp-12], 1
```

neg_outer:

```
    mov     eax, [ebp-12]
    cmp     eax, [ebp-8]
    jge     neg_done

    mov     ecx, eax
    shl     ecx, 3
    lea     edx, [esi + ecx]
    fld     qword ptr [edx]

    mov     ebx, eax
    dec     ebx
```

neg_inner:

```
    cmp     ebx, -1
    jl      neg_insert

    mov     ecx, ebx
    shl     ecx, 3
    lea     edx, [esi + ecx]

    fld     qword ptr [edx]
    fcomp   st(1)
    fstsw   ax
    sahf

    jae     neg_insert

    mov     eax, [edx]
    mov     ecx, [edx+4]
    mov     [edx+8], eax
    mov     [edx+12], ecx
```



```
dec    ebx
jmp    neg_inner
```

neg_insert:

```
inc    ebx
mov    ecx, ebx
shl    ecx, 3
lea    edx, [esi + ecx]
fstp   qword ptr [edx]
```

```
mov    eax, [ebp-12]
inc    eax
mov    [ebp-12], eax
jmp    neg_outer
```

neg_done:

skip_neg_sort:

```
mov    eax, [ebp-4]
mov    edx, pos_count_ptr_
mov    [edx], eax
```

```
mov    eax, [ebp-8]
```

```
pop    edi
pop    esi
pop    ebx
add    esp, 12
mov    esp, ebp
pop    ebp
ret    20
```

sort_worker ENDP

sort_std PROC

```
    push    ebp
    mov     ebp, esp

    push    [ebp+24]
    push    [ebp+20]
    push    [ebp+16]
    push    [ebp+12]
    push    [ebp+8]
    call    sort_worker

    mov     esp, ebp
    pop     ebp
    ret     20
```

sort_std ENDP

sort_c PROC c

```
    push    ebp
    mov     ebp, esp

    push    [ebp+24]
    push    [ebp+20]
    push    [ebp+16]
    push    [ebp+12]
    push    [ebp+8]
    call    sort_worker

    mov     esp, ebp
    pop     ebp
    ret
```

sort_c ENDP

sort_f PROC fastcall

push ebp

mov ebp, esp

push [ebp+16]

push [ebp+12]

push [ebp+8]

push edx

push ecx

call sort_worker

mov esp, ebp

pop ebp

ret 12

sort_f ENDP

**sort_stdcall PROC a_:DWORD, neg_res_:DWORD, pos_res_:DWORD,
length_:DWORD, pos_count_ptr_:DWORD**

push ebp

mov ebp, esp

push pos_count_ptr_

push length_

push pos_res_

push neg_res_

push a_

call sort_worker

mov esp, ebp

pop ebp

ret 20

sort_stdcall ENDP

```
sort_c_arg PROC c pos_count_ptr_:DWORD, neg_res_:DWORD, a_:DWORD,  
length_:DWORD, pos_res_:DWORD
```

```
    push    ebp
```

```
    mov     ebp, esp
```

```
    push    pos_count_ptr_
```

```
    push    length_
```

```
    push    pos_res_
```

```
    push    neg_res_
```

```
    push    a_
```

```
    call    sort_worker
```

```
    mov     esp, ebp
```

```
    pop     ebp
```

```
    ret
```

```
sort_c_arg ENDP
```

```
END
```

```
#include <windows.h>
```

```
#include <iostream>
```

```
#include <random>
```

```
#include <fstream>
```

```
#include <iomanip>
```

```
#include <cstring>
```

```
#include <cmath>
```

```
#pragma comment(lib, "lab8.lib")
```

```
extern "C" __declspec(dllimport)
```

```
int __stdcall sort_std(double* a, double* neg_res, double*
pos_res, int length, int* pos_count);
```

```
extern "C" __declspec(dllimport)
```

```
int __cdecl sort_c(double* a, double* neg_res, double* pos_res,
int length, int* pos_count);
```

```
extern "C" __declspec(dllimport)
```

```
int __fastcall sort_f(double* a, double* neg_res, double*
pos_res, int length, int* pos_count);
```

```
extern "C" __declspec(dllimport)
```

```
int __stdcall sort_stdcall(double* a, double* neg_res, double*
pos_res, int length, int* pos_count);
```

```
extern "C" __declspec(dllimport)
```

```
int __cdecl sort_c_arg(int* pos_count, double* neg_res, double*
a, int length, double* pos_res);
```

```
void insertion_sort_non_decreasing(double* arr, int n) {
```

```
    for (int i = 1; i < n; ++i) {
        double key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            --j;
        }
        arr[j + 1] = key;
    }
}
```

```
}
```

```
void insertion_sort_non_increasing(double* arr, int n) {
```

```
    for (int i = 1; i < n; ++i) {
        double key = arr[i];
```

```

        int j = i - 1;
        while (j >= 0 && arr[j] < key) {
            arr[j + 1] = arr[j];
            --j;
        }
        arr[j + 1] = key;
    }
}

```

```

int sort_cpp(double* a, double* neg_res, double* pos_res, int
length, int* pos_count) {
    int pos_cnt = 0;
    int neg_cnt = 0;

    for (int i = 0; i < length; ++i) {
        double x = a[i];
        if (x < 0.0) {
            neg_res[neg_cnt++] = x;
        } else {
            pos_res[pos_cnt++] = x;
        }
    }

    insertion_sort_non_decreasing(pos_res, pos_cnt);
    insertion_sort_non_increasing(neg_res, neg_cnt);

    *pos_count = pos_cnt;
    return neg_cnt;
}

```

```

int sort_cpp_asm(double* a, double* neg_res, double* pos_res, int
length, int* pos_count) {
    int pos_cnt = 0;
    int neg_cnt = 0;

```

```

__asm {
    push ebx
    push esi
    push edi

    mov  esi, a          ; a
    mov  ebx, neg_res    ; neg_res
    mov  edi, pos_res    ; pos_res
    mov  ecx, length     ; length
    xor  edx, edx        ; i = 0

split_loop_cpp_asm:
    cmp  edx, ecx
    jge  split_done_cpp_asm

    mov  eax, [esi + edx*8 + 4]
    test eax, 80000000h
    jnz  store_neg_cpp_asm

    mov  eax, pos_cnt
    mov  ecx, eax
    shl  ecx, 3

    mov  eax, [esi + edx*8]
    mov  DWORD PTR [edi + ecx], eax
    mov  eax, [esi + edx*8 + 4]
    mov  DWORD PTR [edi + ecx + 4], eax

    inc  pos_cnt
    jmp  next_i_cpp_asm

store_neg_cpp_asm:

```

```

mov    eax, neg_cnt
mov    ecx, eax
shl    ecx, 3

mov    eax, [esi + edx*8]
mov    DWORD PTR [ebx + ecx], eax
mov    eax, [esi + edx*8 + 4]
mov    DWORD PTR [ebx + ecx + 4], eax

inc    neg_cnt

```

```

next_i_cpp_asm:

```

```

    inc    edx
    jmp    split_loop_cpp_asm

```

```

split_done_cpp_asm:

```

```

    pop    edi
    pop    esi
    pop    ebx

```

```

}

```

```

insertion_sort_non_decreasing(pos_res, pos_cnt);
insertion_sort_non_increasing(neg_res, neg_cnt);

```

```

*pos_count = pos_cnt;
return neg_cnt;

```

```

}

```

```

void print_array(const char* name, const double* arr, int n) {
    std::cout << name << " (n = " << n << "): ";
    for (int i = 0; i < n; ++i) {
        std::cout << std::fixed << std::setprecision(1) << arr[i]
<< " ";
    }
}

```



```

        std::cout << "\n";
    }

bool compare_arrays(const double* a, const double* b, int n,
double eps = 1e-9) {
    for (int i = 0; i < n; ++i) {
        if (std::fabs(a[i] - b[i]) > eps) return false;
    }
    return true;
}

int main() {
    setlocale(LC_ALL, "Russian");

    std::ofstream f_std("stdcall.csv");
    std::ofstream f_c("cdecl.csv");
    std::ofstream f_f("fastcall.csv");
    std::ofstream f_cpp("cpp.csv");
    std::ofstream f_cpp_asm("cpp_asm.csv");

    {
        const int n = 10;
        double a[n] = { 1, 3, 4, -5, 7, -2, -1, 3, 5, -5 };

        double pos1[n] = {}, neg1[n] = {};
        double pos2[n] = {}, neg2[n] = {};
        double pos3[n] = {}, neg3[n] = {};
        double pos4[n] = {}, neg4[n] = {};
        double pos5[n] = {}, neg5[n] = {};
        double pos6[n] = {}, neg6[n] = {};
        double pos7[n] = {}, neg7[n] = {};

        int pc1 = 0, nc1 = 0;
        int pc2 = 0, nc2 = 0;
    }
}

```

```

int pc3 = 0, nc3 = 0;
int pc4 = 0, nc4 = 0;
int pc5 = 0, nc5 = 0;
int pc6 = 0, nc6 = 0;
int pc7 = 0, nc7 = 0;

nc1 = sort_std(a, neg1, pos1, n, &pc1);
nc2 = sort_c(a, neg2, pos2, n, &pc2);
nc3 = sort_f(a, neg3, pos3, n, &pc3);
nc4 = sort_stdcall(a, neg4, pos4, n, &pc4);
nc5 = sort_c_arg(&pc5, neg5, a, n, pos5);      //
прототип из задания
nc6 = sort_cpp(a, neg6, pos6, n, &pc6);
nc7 = sort_cpp_asm(a, neg7, pos7, n, &pc7);

std::cout << "Исходный массив: ";
for (int i = 0; i < n; ++i) std::cout << a[i] << " ";
std::cout << "\n\n";

print_array("pos_std    ", pos1, pc1);
print_array("neg_std    ", neg1, nc1);
print_array("pos_c      ", pos2, pc2);
print_array("neg_c      ", neg2, nc2);
print_array("pos_f      ", pos3, pc3);
print_array("neg_f      ", neg3, nc3);
print_array("pos_stdcl  ", pos4, pc4);
print_array("neg_stdcl  ", neg4, nc4);
print_array("pos_c_arg  ", pos5, pc5);
print_array("neg_c_arg  ", neg5, nc5);
print_array("pos_cpp    ", pos6, pc6);
print_array("neg_cpp    ", neg6, nc6);
print_array("pos_cpp_asm", pos7, pc7);
print_array("neg_cpp_asm", neg7, nc7);
}

```

```

std::default_random_engine gen;
std::uniform_int_distribution<int> dist(-10000, 10000);

for (int n = 10000; n <= 30000; n += 2000) {
    double* a          = new double[n];
    double* a_copy1 = new double[n];
    double* a_copy2 = new double[n];
    double* a_copy3 = new double[n];
    double* a_copy4 = new double[n];

    double* pos = new double[n];
    double* neg = new double[n];

    for (int i = 0; i < n; ++i) a[i] =
static_cast<double>(dist(gen));

    std::memcpy(a_copy1, a, sizeof(double) * n);
    std::memcpy(a_copy2, a, sizeof(double) * n);
    std::memcpy(a_copy3, a, sizeof(double) * n);
    std::memcpy(a_copy4, a, sizeof(double) * n);

    int pc, nc;
    DWORD t1, t2;

    t1 = GetTickCount();
    nc = sort_std(a, neg, pos, n, &pc);
    t2 = GetTickCount();
    f_std << n << ";" << (t2 - t1) << "\n";

    t1 = GetTickCount();
    nc = sort_c(a_copy1, neg, pos, n, &pc);
    t2 = GetTickCount();
    f_c << n << ";" << (t2 - t1) << "\n";

```

```
t1 = GetTickCount();
nc = sort_f(a_copy2, neg, pos, n, &pc);
t2 = GetTickCount();
f_f << n << ";" << (t2 - t1) << "\n";
```

```
t1 = GetTickCount();
nc = sort_cpp(a_copy3, neg, pos, n, &pc);
t2 = GetTickCount();
f_cpp << n << ";" << (t2 - t1) << "\n";
```

```
t1 = GetTickCount();
nc = sort_cpp_asm(a_copy4, neg, pos, n, &pc);
t2 = GetTickCount();
f_cpp_asm << n << ";" << (t2 - t1) << "\n";
```

```
delete[] a;
delete[] a_copy1;
delete[] a_copy2;
delete[] a_copy3;
delete[] a_copy4;
delete[] pos;
delete[] neg;
}
```

```
f_std.close();
f_c.close();
f_f.close();
f_cpp.close();
f_cpp_asm.close();
```

```
std::cout << "\nГотово. CSV-файлы с временем готовы, можно  
строить графики.\n";
```

```
std::cout << "Нажмите Enter для выхода...";
std::cin.get();
```

```

    return 0;
}

```



Вывод: изучила способы вызова подпрограмм, написанных на разных языках программирования посредством dll-библиотек.